Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Ing3 – Web Dev I

TP6 : Sessions & Cookies

Sécurisation du catalogue de films

Objectif du TP

Ce TP est la suite du TP5. Vous allez sécuriser votre application de gestion de films en ajoutant un système d'authentification par sessions PHP. Vous explorerez également les cookies pour en comprendre le fonctionnement, les cas d'usage, et les différences fondamentales avec les sessions.

Prérequis : Le projet TP5 doit être fonctionnel (connexion PDO, CRUD films opérationnel), la solution est disponible dans la plateforme `connect.uir.ac.ma`. Vous travaillerez dans le même dossier `htdocs/TP5` – aucune nouvelle base de données n'est requise pour ce TP.

Étape 1 : Création de la table `utilisateurs`

— Dans phpMyAdmin, ajoutez la table `utilisateurs` à la base `CinemaBD` selon la structure suivante :

#	Nom	Type	Interclassement	Null	Extra
1	`id`	`INT(11)`	–	Non	`AUTO_INCREMENT`
2	`login`	`VARCHAR(60)`	`utf8mb4_general_ci`	Non	
3	`mot_de_passe`	`VARCHAR(255)`	`utf8mb4_general_ci`	Non	

— Insérez manuellement **un utilisateur de test** via l'onglet SQL de phpMyAdmin. Le mot de passe doit être haché avec `password_hash()`, exécutez d'abord le script PHP ci-dessous pour obtenir le hash, puis copiez-le dans la requête INSERT :

```php
<?php
// Affiche le hash a copier dans phpMyAdmin
echo password_hash("cinema2025", PASSWORD_DEFAULT);
```

Listing 1 – generer_hash.php – à exécuter une seule fois

```sql
INSERT INTO utilisateurs (login, mot_de_passe)
VALUES ('admin', '$2y$10$VotreHashIciCopieDepuisGenererHash');
```

Listing 2 – Requête SQL d'insertion à adapter avec le hash obtenu

Pourquoi `VARCHAR(255)` pour le mot de passe ? La fonction `password_hash()` génère un hash bcrypt d'environ 60 caractères, mais la taille peut varier selon l'algorithme. `VARCHAR(255)` est la taille recommandée pour rester compatible avec les évolutions futures de PHP.

Étape 2 : Formulaire de connexion (`login.php`)

- Créez le fichier `login.php` dans votre projet TP5. Ce fichier doit afficher un formulaire HTML contenant :
  - Un champ `login` (type `text`),
  - Un champ `mot_de_passe` (type `password`),
  - Un bouton de soumission.
- Le formulaire envoie les données en **POST** vers `login.php`.

Comment fonctionne une session PHP ? Une session permet de mémoriser des informations côté serveur entre plusieurs requêtes HTTP. Pour l'utiliser, trois règles à respecter :

- **Démarrer** la session avec `session_start()` – obligatoirement **avant tout affichage HTML**, en première ligne du fichier.
- **Stocker** une information : `$_SESSION['cle'] = valeur;`
- **Lire** une information : `$_SESSION['cle']` (disponible dans tous les fichiers qui appellent `session_start()`).

- En haut du fichier `login.php`, ajoutez `session_start()`.
- Vérifiez si l'utilisateur est **déjà connecté** en testant l'existence de `$_SESSION['utilisateur']` avec `isset()`, si c'est le cas, redirigez-le immédiatement vers `catalogue.php` avec `header('Location: catalogue.php')` suivi d'un `exit`.
- Déclarez une variable `$erreur = ''` qui servira à afficher un message en cas d'identifiants incorrects.
- Vérifiez si le formulaire a été soumis en testant `$_SERVER['REQUEST_METHOD'] === 'POST'`, puis récupérez les champs `$_POST['login']` et `$_POST['mot_de_passe']` dans deux variables (utilisez `trim()` pour supprimer les espaces).
- Dans le HTML, affichez le contenu de `$erreur` si elle n'est pas vide.

Étape 3 : Vérification des identifiants et ouverture de session

- Dans le fichier `login.php`, implémentez la vérification en deux temps :
  1. Recherchez l'utilisateur en base via une **requête PDO préparée** sur le champ `login` (Faite appel au fichier `connexion.php`).
  2. Si un enregistrement est trouvé, vérifiez le mot de passe soumis avec `password_verify()`.
- En cas de succès :
  - Stockez le login dans `$_SESSION['utilisateur']`,
  - Redirigez vers `catalogue.php` avec `header('Location: ...')`.
- En cas d'échec : affectez un message d'erreur à `$erreur`.

Étape 4 : Protection des pages (garde de session)

- Ajoutez au **tout début** de chaque fichier PHP du catalogue (`catalogue.php`, `ajout.php`, `modifierFilm.php`, `supprimerFilm.php`) le bloc suivant :

```php
<?php
session_start();
if (!isset($_SESSION['utilisateur'])) {
    header('Location: login.php');
    exit;
}
```

Listing 3 – Garde de session à placer en premiere ligne de chaque page protegee

— Testez en ouvrant directement `catalogue.php` dans le navigateur sans être connecté : vous devez être automatiquement redirigé vers `login.php`.
— Connectez-vous ensuite avec les identifiants créés à l'étape 1 et vérifiez que l'accès est accordé.

Rappel cycle de vie d'une session :

— `session_start()` doit être appelé **avant tout affichage HTML** dans chaque fichier qui utilise `$_SESSION`.
— La session est identifiée côté client par un cookie de session (`PHPSESSID`), les données elles-mêmes restent sur le serveur.
— La session se termine à la fermeture du navigateur ou via `session_destroy()`.

Étape 5 : Affichage du nom d'utilisateur et déconnexion

— Dans `catalogue.php`, affichez en haut de page un message de bienvenue utilisant la variable `login` de la session cree, ainsi qu'un lien vers `logout.php`.
— Créez le fichier `logout.php` qui :
  1. Démarre la session (`session_start()`),
  2. Efface toutes les variables de session (`session_unset()`),
  3. Détruit la session (`session_destroy()`),
  4. Redirige vers `login.php`.

Étape 6 : Introduction aux cookies – préférence d'affichage

Dans cette étape, vous allez utiliser un **cookie** pour mémoriser une préférence d'affichage de l'utilisateur : le nombre de films affichés par page. Contrairement à la session (données côté serveur), le cookie est stocké **dans le navigateur du client** et persiste même après fermeture du navigateur.

— Ajoutez dans `catalogue.php` un formulaire simple permettant à l'utilisateur de choisir **5, 10 ou tous** les films par page et qui renvoie en utilisant la methode `GET`, et qui cree le cookie de preference.
— Modifiez la requête SQL dans `getAll()` pour appliquer une clause `LIMIT` selon la valeur de `$parPage` (si `$parPage === 0`, ne pas ajouter de `LIMIT`).
— Vérifiez dans les outils développeur du navigateur (onglet **Application** → **Cookies**) que le cookie `film_par_page` est bien créé.
— Fermez et rouvrez le navigateur : la préférence doit être **mémorisée** (comportement impossible avec une session seule).

Rappel important : `session_start()` et `setcookie()` doivent obligatoirement être appelés **avant tout envoi de contenu HTML** (y compris les espaces et sauts de ligne avant `<?php`). Un *header already sent* est l'erreur la plus fréquente en travaillant avec sessions et cookies.

Devoir – Gestion de rôles utilisateurs

Contexte

Pour aller plus loin, vous allez enrichir le système d'authentification avec une gestion de **rôles** : un utilisateur de type `admin` peut ajouter, modifier et supprimer des films ; un utilisateur de type `visiteur` peut uniquement consulter le catalogue.

Travail demandé

1. **Ajout du champ `role` à la table `utilisateurs`**
   Ajoutez la colonne suivante à la table `utilisateurs` via phpMyAdmin :

#	Nom	Type	Défaut	Valeurs possibles
4	`role`	`ENUM('admin','visiteur')`	`'visiteur'`	–

   Insérez un second utilisateur avec le rôle `visiteur`. Mettez à jour l'utilisateur `admin` pour lui attribuer le rôle `admin`.

2. **Stockage du rôle en session**
   Modifiez `login.php` pour stocker également le rôle en session lors de la connexion.

3. **Classe `Auth` : vérification de rôle**
   Créez un fichier `Auth.php` contenant la classe suivante à compléter :

```php
<?php
class Auth {
    /**
     * Redirige vers login.php si aucune session active.
     */
    public static function requireLogin(): void {
        // TODO 1 : Demarrer la session si elle n'est pas encore active
        //          Astuce : utilisez session_status() === PHP_SESSION_NONE
        //          pour verifier avant d'appeler session_start()

        // TODO 2 : Verifier si $_SESSION['utilisateur'] existe avec isset()
        //          Si NON -> rediriger vers login.php
    }

    /**
     * Redirige vers catalogue.php avec un message d'erreur
     * si le role en session ne correspond pas a $roleRequis.
     */
    public static function requireRole(string $roleRequis): void {
        // TODO 1 : Demarrer la session si elle n'est pas encore active
        //          (meme logique que dans requireLogin)

        // TODO 2 : Recuperer le role stocke dans $_SESSION['role']
        //          Utilisez l'operateur ?? pour definir 'visiteur' par defaut
        //          si la cle n'existe pas : $_SESSION['role'] ?? 'visiteur'

        // TODO 3 : Comparer le role actuel avec $roleRequis
        //          Si ils sont differents :
        //          - Stocker un message d'erreur dans $_SESSION['erreur_acces']
        //          - Rediriger vers catalogue.php avec header()
        //          - Terminer avec exit
    }

    /**
     * Retourne true si l'utilisateur connecte est admin.
     */
    public static function isAdmin(): bool {
        // TODO 1 : Demarrer la session si elle n'est pas encore active
```

```
39
40          // TODO 2 : Verifier que $_SESSION['role'] existe ET vaut '
            admin'
41          //           Utilisez isset() ET l'operateur === pour la
            comparaison
42          //           Retournez le resultat directement avec return
43      }
44  }
```

Listing 4 – Auth.php squelette fourni

4. **Protection des actions d'écriture**

   Les pages `ajout.php`, `modifierFilm.php` et `supprimerFilm.php` doivent être accessibles uniquement aux administrateurs. Pour chacune de ces pages :

   — Ajoutez `require_once 'Auth.php';` pour charger la classe `Auth`.
   — Appelez `Auth::requireLogin()` pour vérifier qu'un utilisateur est connecté.
   — Appelez `Auth::requireRole('admin')` pour vérifier que l'utilisateur connecté est bien un administrateur.
   — Ces deux appels doivent être placés **avant** tout autre code (après `session_start()` et `require_once`).

   > **Pourquoi deux appels distincts ?** `requireLogin()` vérifie qu'une session est ouverte – elle redirige vers `login.php` si l'utilisateur n'est pas connecté du tout. `requireRole('admin')` vérifie ensuite le rôle – elle redirige vers `catalogue.php` si l'utilisateur est connecté mais n'est pas admin. Les deux cas d'erreur sont ainsi traités séparément et clairement.

5. **Affichage conditionnel dans `catalogue.php`**

   Un visiteur connecté ne doit pas voir les liens *Modifier* et *Supprimer* dans le tableau des films. Modifiez `catalogue.php` comme suit :

   — Dans la boucle d'affichage des films, entourez les liens *Modifier* et *Supprimer* d'une condition PHP : appelez `Auth::isAdmin()` et affichez ces liens **uniquement** si la méthode retourne `true`.
   — Dans la barre en haut de page, affichez un message de bienvenue indiquant le login de l'utilisateur connecté (`$_SESSION['utilisateur']`) ainsi que son rôle (`$_SESSION['role']`) entre parenthèses.

6. **Cookie de dernière connexion**

   Dans `login.php`, après une authentification réussie, enregistrez un cookie `derniere_connexion` contenant la date et l'heure de la connexion (format `d/m/Y H:i`), avec une durée de vie de **30 jours**.

   Dans `catalogue.php`, affichez un message du type : *« Dernière connexion : 18/03/2026 à 14 :32 »* si le cookie existe.

> **Rappel signatures imposées :** les signatures des méthodes de la classe `Auth` (paramètres, types de retour) ne doivent pas être modifiées. Toute implémentation qui ne respecte pas la signature exacte sera considérée comme incorrecte, même si le code fonctionne.